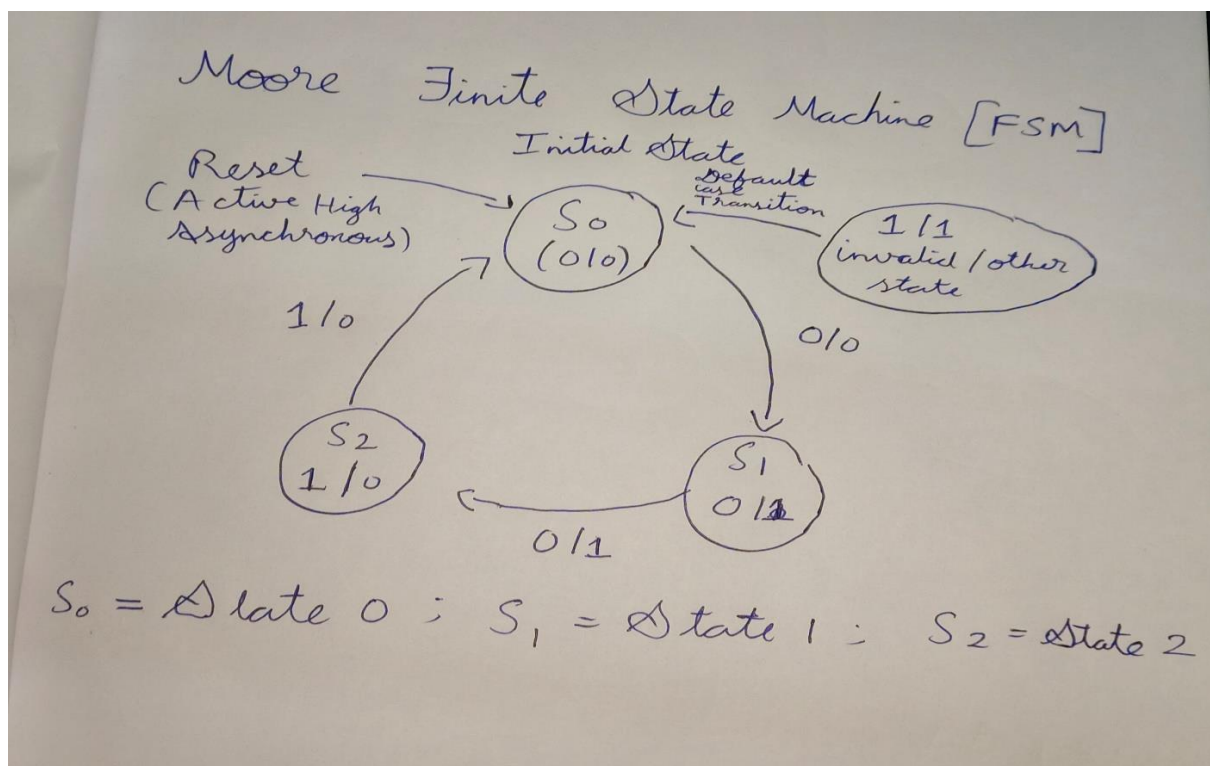


Finite State Machine

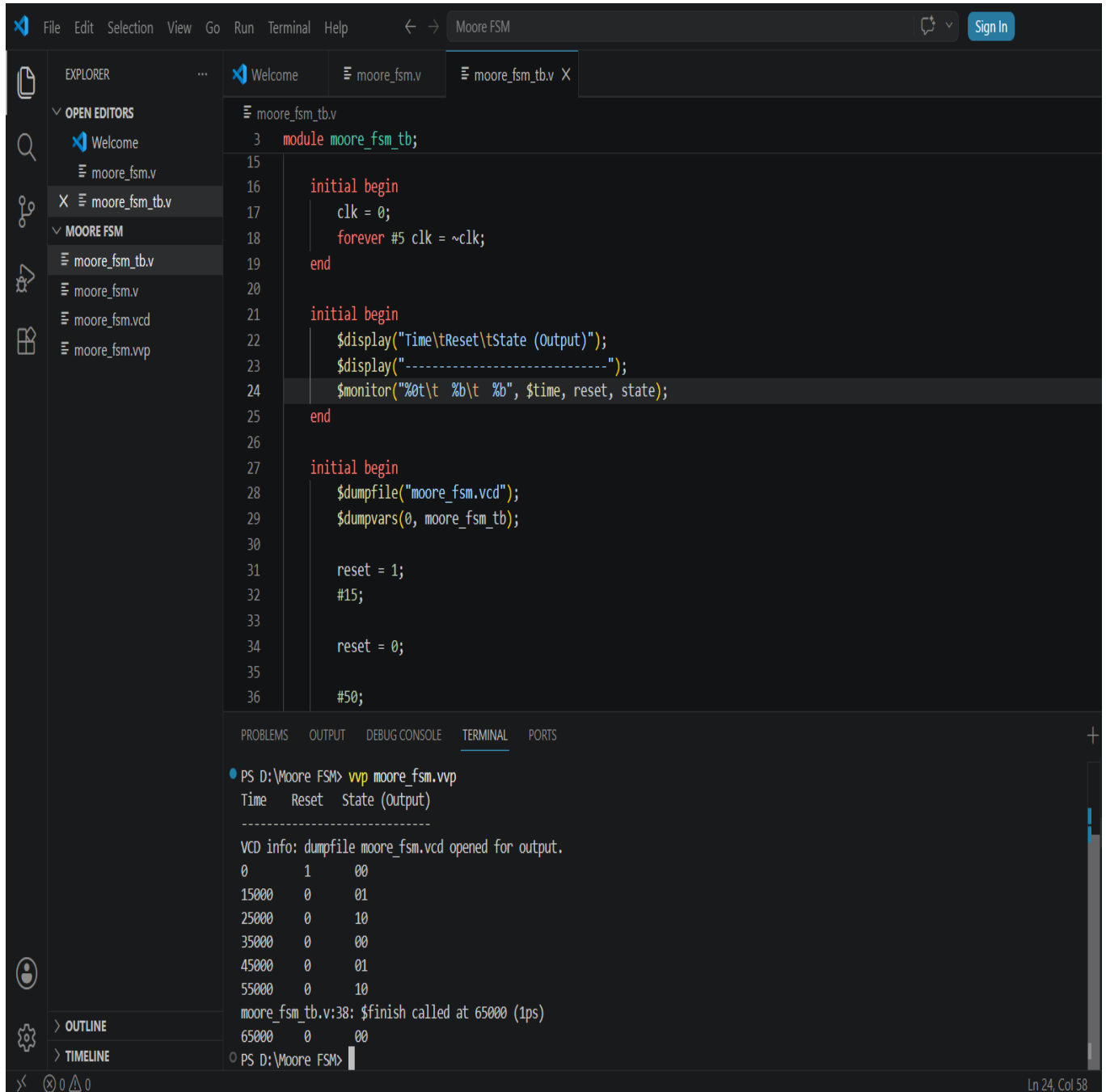
A Finite State Machine (FSM) is an abstract mathematical model of computation used to design both computer programs and sequential logic circuits. Simply put, it is a system that can only exist in exactly one of a limited (finite) number of predefined states at any given time. The machine transitions from one state to another in response to external inputs.

Moore Finite State Machine

A Moore Finite State Machine (FSM) is a digital logic circuit where the outputs depend solely on the current state of the machine. In a Moore machine, the external inputs only determine what the next state will be. They do not directly or immediately affect the output. If an input signal spikes or glitches momentarily between clock cycles, the output will not change. This makes Moore machines very stable.



Output of Moore Finite State Machine



The screenshot displays an IDE interface with a dark theme. The Explorer panel on the left shows a project named 'MOORE FSM' containing files: 'moore_fsm.v', 'moore_fsm_tb.v', 'moore_fsm.vcd', and 'moore_fsm.vvp'. The main editor window shows the 'moore_fsm_tb.v' file with the following Verilog code:

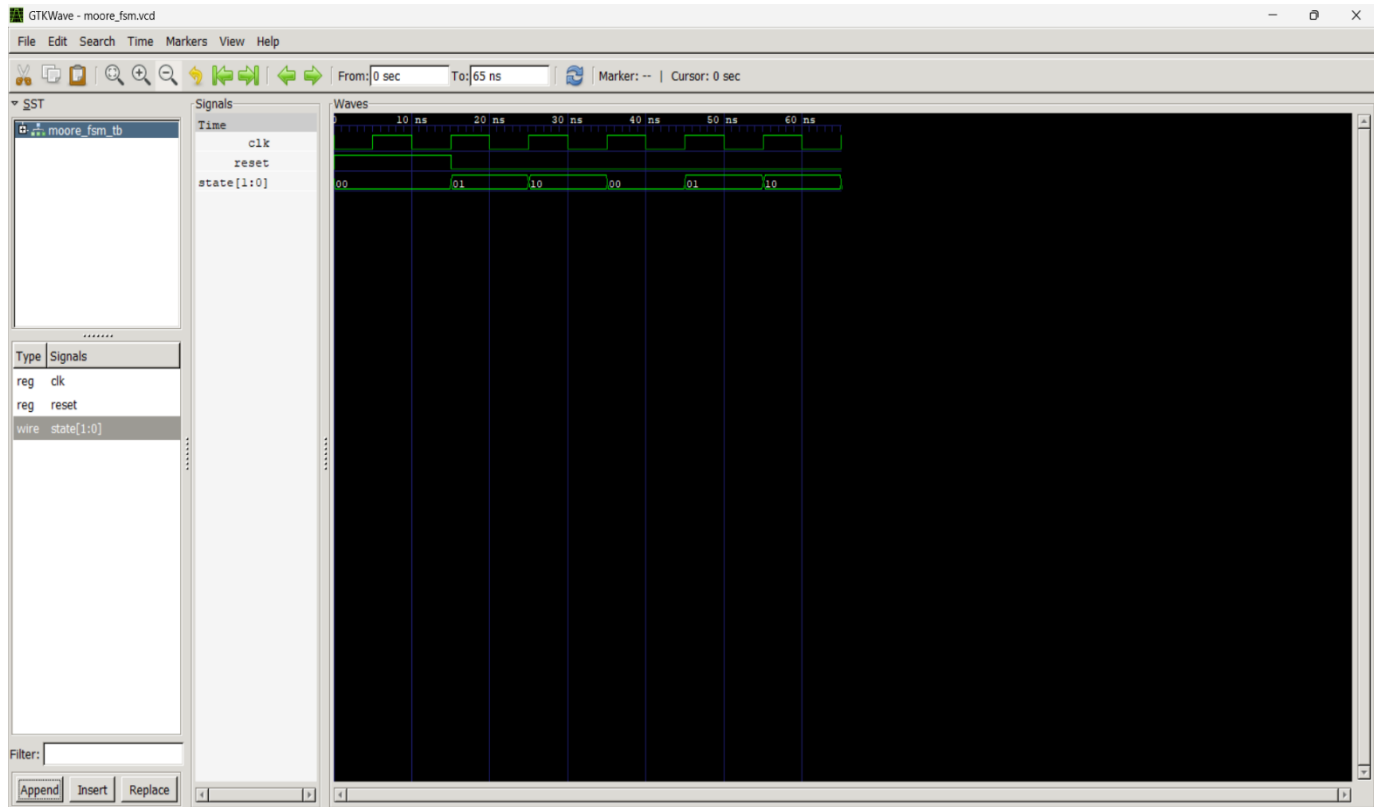
```
3 module moore_fsm_tb;
15
16   initial begin
17     clk = 0;
18     forever #5 clk = ~clk;
19   end
20
21   initial begin
22     $display("Time\tReset\tState (Output)");
23     $display("-----");
24     $monitor("%0t\t %b\t %b", $time, reset, state);
25   end
26
27   initial begin
28     $dumpfile("moore_fsm.vcd");
29     $dumpvars(0, moore_fsm_tb);
30
31     reset = 1;
32     #15;
33
34     reset = 0;
35
36     #50;
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS D:\Moore FSM> vvp moore_fsm.vvp
Time  Reset  State (Output)
-----
VCD info: dumpfile moore_fsm.vcd opened for output.
0      1      00
15000  0      01
25000  0      10
35000  0      00
45000  0      01
55000  0      10
moore_fsm_tb.v:38: $finish called at 65000 (1ps)
65000  0      00
PS D:\Moore FSM>
```

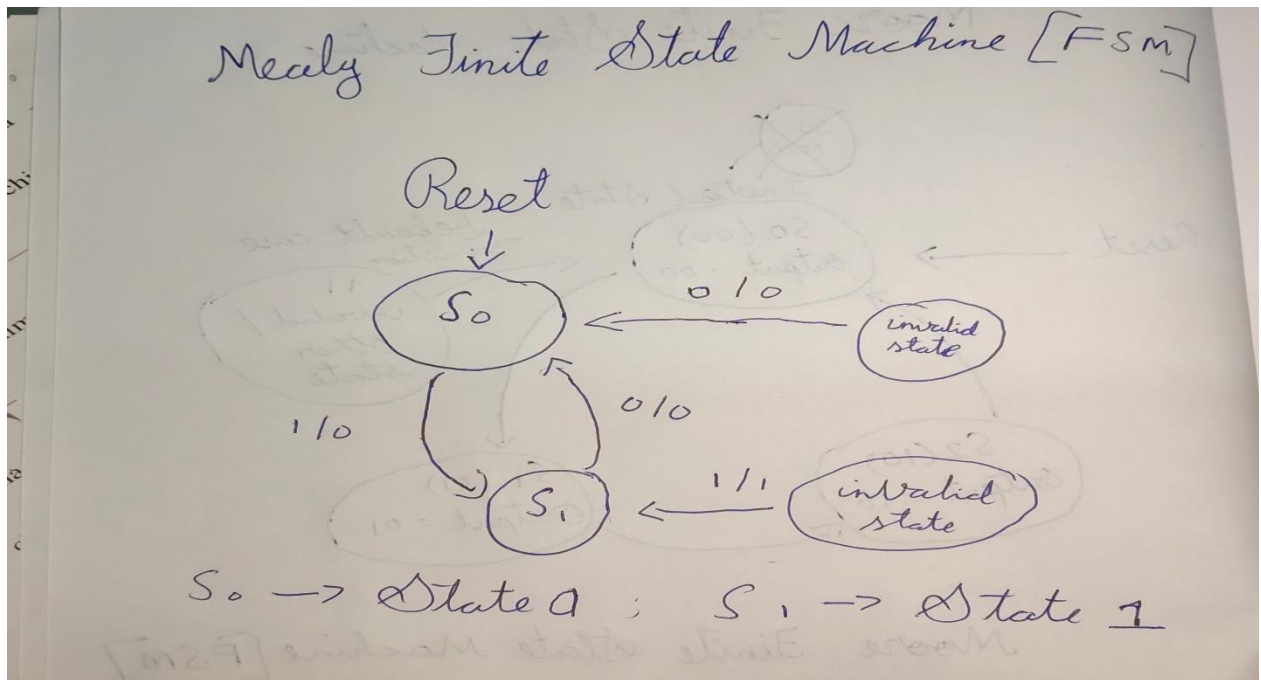
The status bar at the bottom right indicates 'Ln 24, Col 58'.

Waveform of Moore Finite State Machine



Mealy Finite State Machine

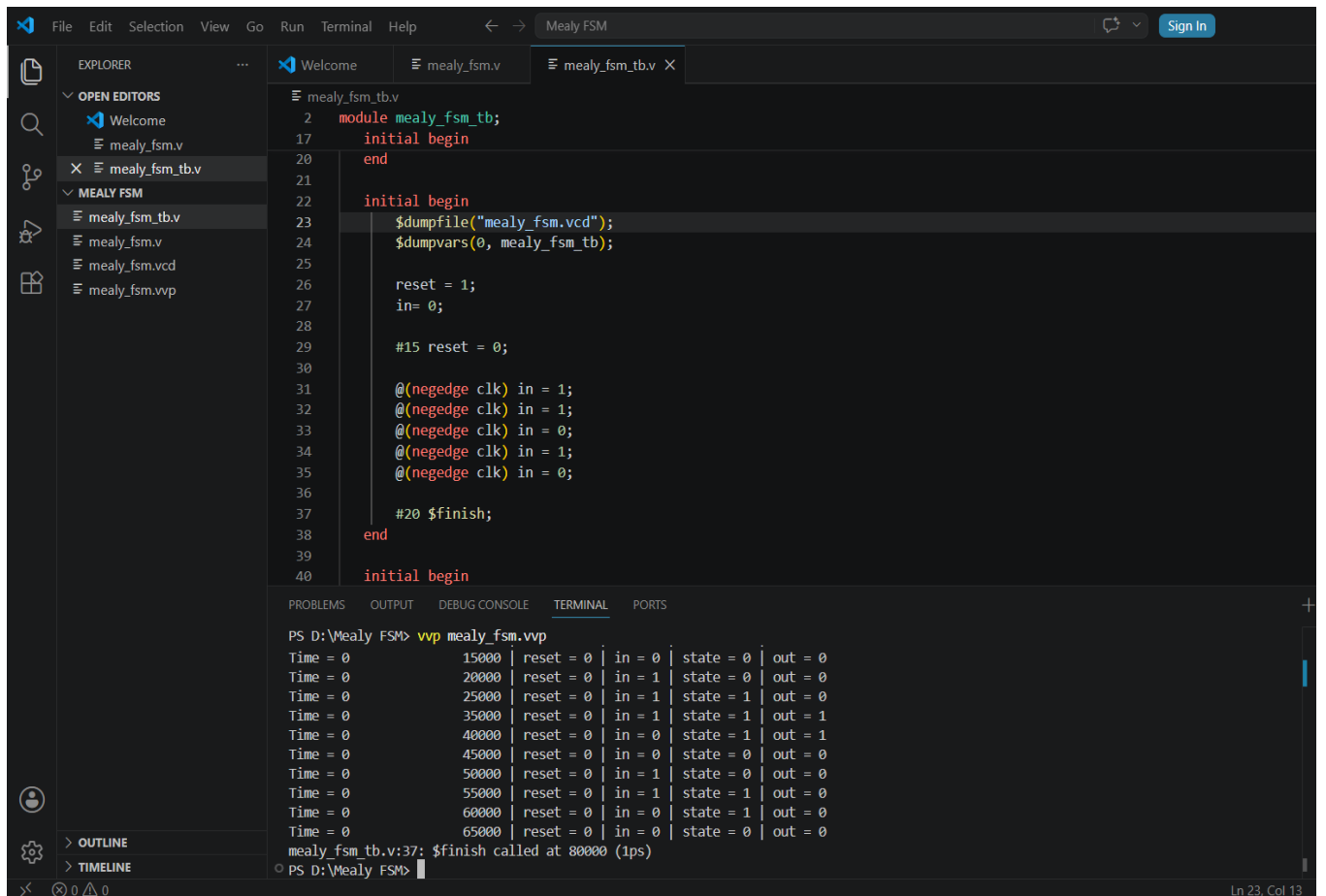
A Mealy Finite State Machine (FSM) is a digital logic model where the outputs are determined by both the current state and the current inputs. Mealy machines typically require fewer states than Moore machines to implement the same logic. A single state can produce multiple different outputs depending on which input triggers the transition.



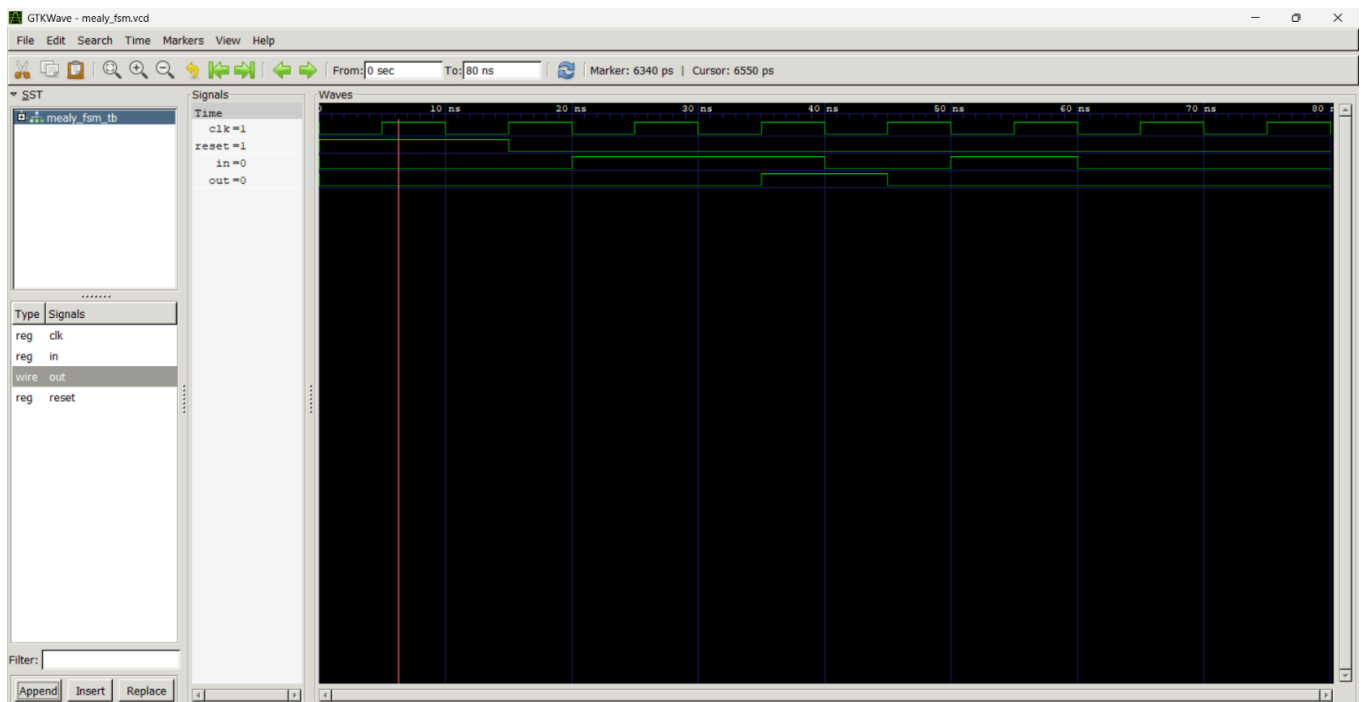
Output of Mealy Finite State Machine

```

File Edit Selection View Go Run Terminal Help Mealy FSM Sign In
EXPLORER
  OPEN EDITORS
    Welcome
    mealy_fsm.v
    mealy_fsm_tb.v
  MEALY FSM
    mealy_fsm_tb.v
    mealy_fsm.v
    mealy_fsm.vcd
    mealy_fsm.vvp
  mealy_fsm_tb.v
2 module mealy_fsm_tb;
17 initial begin
20 end
21
22 initial begin
23   $dumpfile("mealy_fsm.vcd");
24   $dumpvars(0, mealy_fsm_tb);
25
26   reset = 1;
27   in = 0;
28
29   #15 reset = 0;
30
31   @(negedge clk) in = 1;
32   @(negedge clk) in = 1;
33   @(negedge clk) in = 0;
34   @(negedge clk) in = 1;
35   @(negedge clk) in = 0;
36
37   #20 $finish;
38 end
39
40 initial begin
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Mealy FSM> iverilog -o mealy_fsm.vvp mealy_fsm.v mealy_fsm_tb.v
PS D:\Mealy FSM> vvp mealy_fsm.vvp
VCD info: dumpfile mealy_fsm.vcd opened for output.
Time = 0 0 | reset = 1 | in = 0 | state = 0 | out = 0
Time = 0 15000 | reset = 0 | in = 0 | state = 0 | out = 0
Time = 0 20000 | reset = 0 | in = 1 | state = 0 | out = 0
Time = 0 25000 | reset = 0 | in = 1 | state = 1 | out = 0
Time = 0 35000 | reset = 0 | in = 1 | state = 1 | out = 1
Time = 0 40000 | reset = 0 | in = 0 | state = 1 | out = 1
Time = 0 45000 | reset = 0 | in = 0 | state = 0 | out = 0
Time = 0 50000 | reset = 0 | in = 1 | state = 0 | out = 0
Time = 0 55000 | reset = 0 | in = 1 | state = 1 | out = 0
Time = 0 60000 | reset = 0 | in = 0 | state = 1 | out = 0
Ln 23, Col 13
  
```

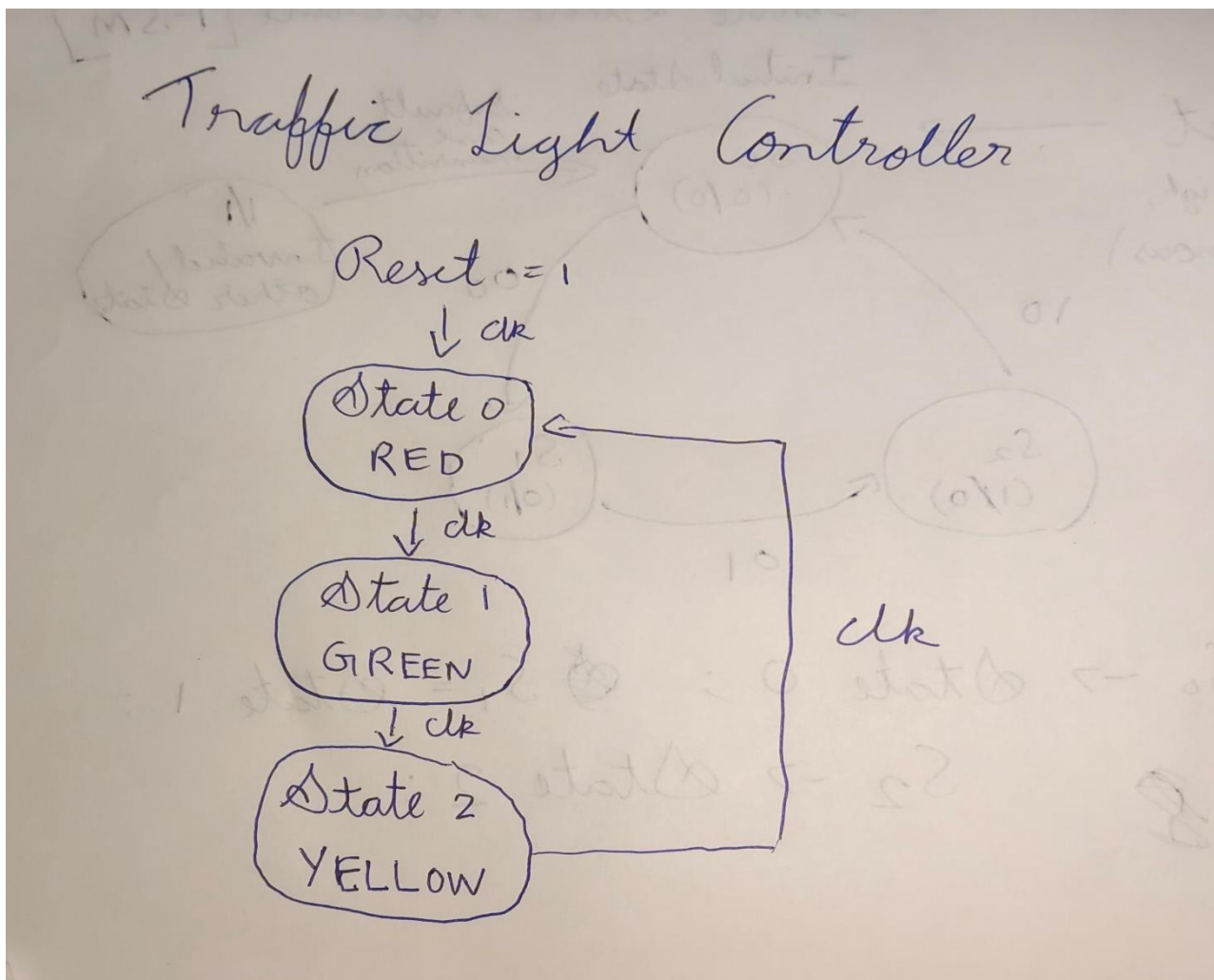


Waveform of Mealy Finite State Machine



Traffic Light Controller

- A Traffic Light Controller is a real-world application of a Finite State Machine (FSM).
- It is a digital logic system designed to sequence through a specific set of states (the lights) safely and predictably.
- The system has distinct "modes" it can be in. For your project, these are State 0 (Red), State 1 (Green), and State 2 (Yellow). The controller evaluates its current state and moves to the next state every time the clock ticks (the "posedge" or positive edge).
- Depending strictly on what state the machine is currently in, it outputs a specific binary signal to turn the physical LEDs on or off.



Output of Traffic Light Controller

The screenshot shows an IDE with the following components:

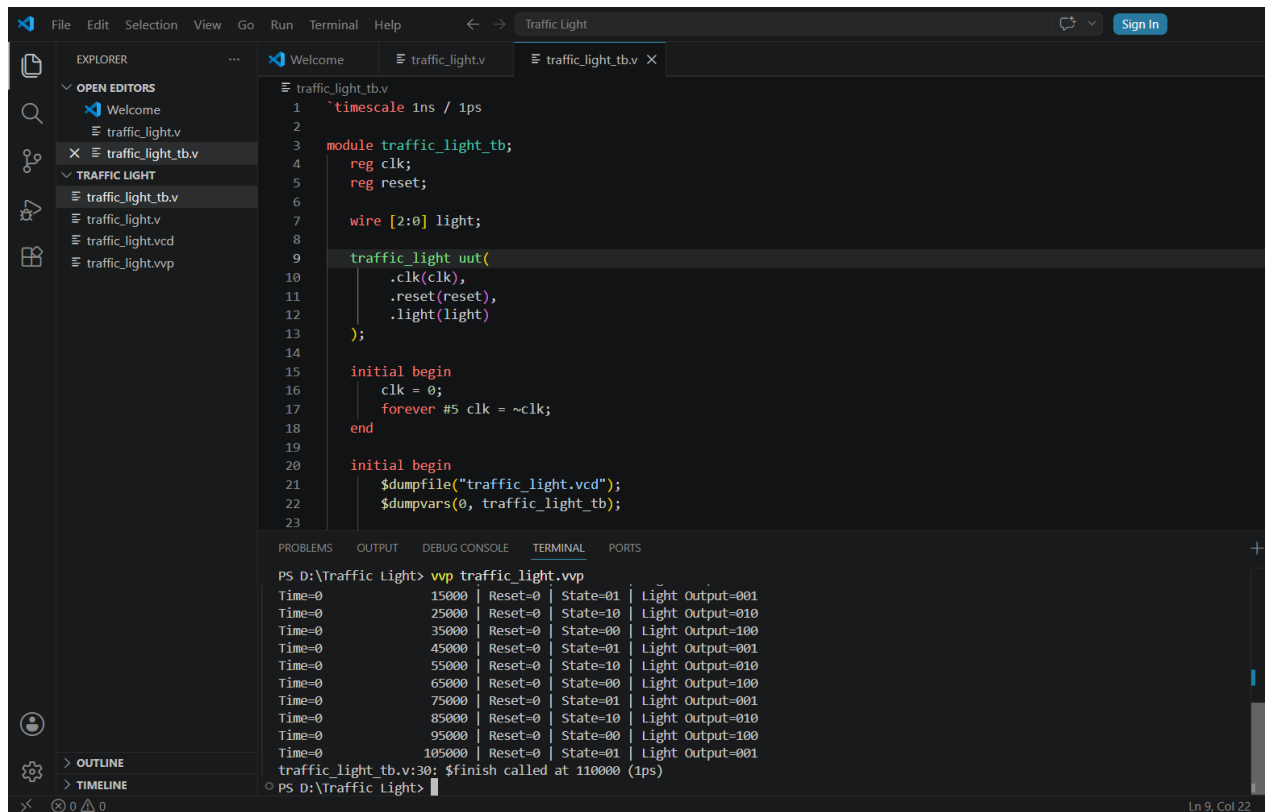
- EXPLORER:** Shows a project structure with files: `Welcome`, `traffic_light.v`, `traffic_light_tb.v`, `traffic_light.vcd`, and `traffic_light.vvp`.
- EDITOR:** Displays the Verilog code for `traffic_light_tb.v`.
- TERMINAL:** Shows the command `vvp traffic_light.vvp` and the resulting output.

Verilog Code (traffic_light_tb.v):

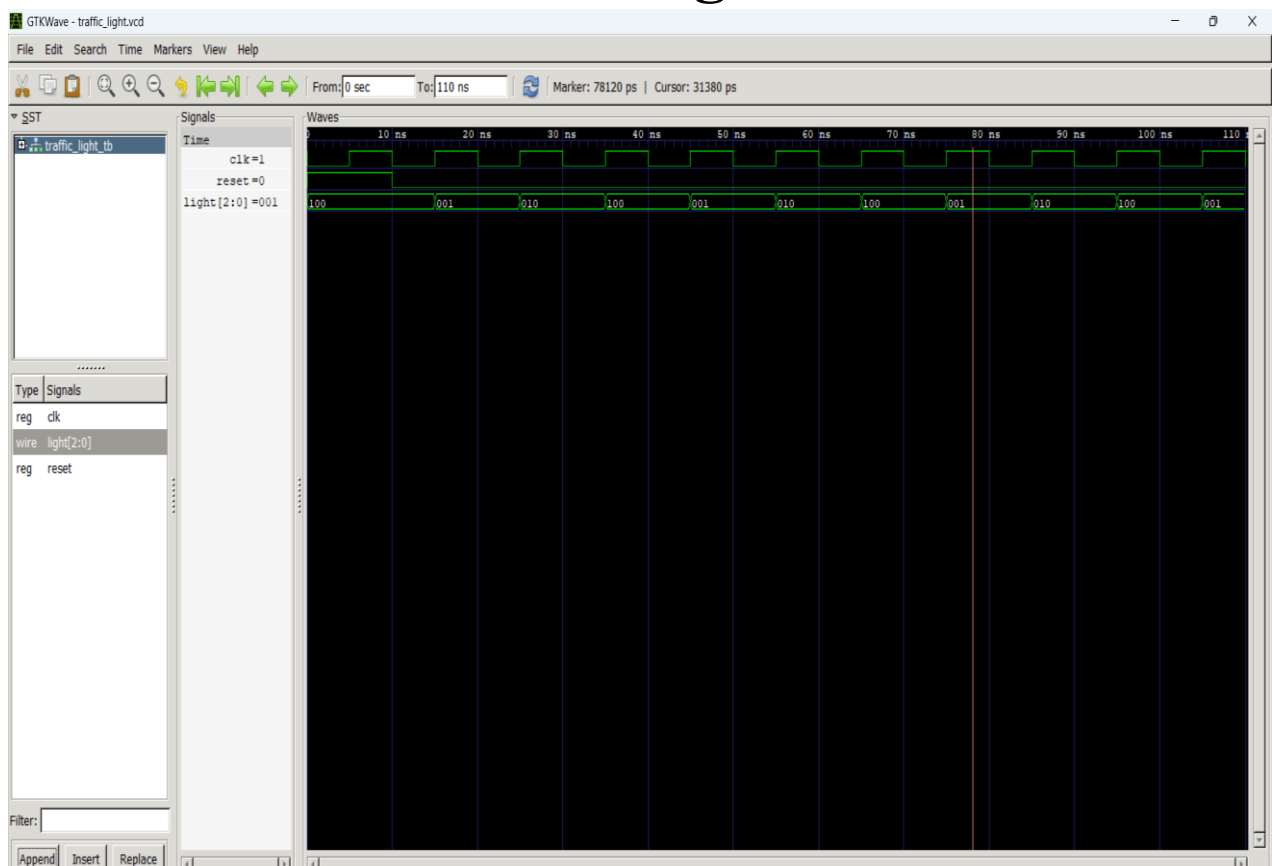
```
1 `timescale 1ns / 1ps
2
3 module traffic_light_tb;
4     reg clk;
5     reg reset;
6
7     wire [2:0] light;
8
9     traffic_light uut(
10         .clk(clk),
11         .reset(reset),
12         .light(light)
13     );
14
15     initial begin
16         clk = 0;
17         forever #5 clk = ~clk;
18     end
19
20     initial begin
21         $dumpfile("traffic_light.vcd");
22         $dumpvars(0, traffic_light_tb);
23     end
```

Terminal Output:

```
PS D:\Traffic Light> vvp traffic_light.vvp
VCD info: dumpfile traffic_light.vcd opened for output.
Time=0          0 | Reset=1 | State=00 | Light Output=100
Time=0        10000 | Reset=0 | State=00 | Light Output=100
Time=0        15000 | Reset=0 | State=01 | Light Output=001
Time=0        25000 | Reset=0 | State=10 | Light Output=010
Time=0        35000 | Reset=0 | State=00 | Light Output=100
Time=0        45000 | Reset=0 | State=01 | Light Output=001
Time=0        55000 | Reset=0 | State=10 | Light Output=010
Time=0        65000 | Reset=0 | State=00 | Light Output=100
Time=0        75000 | Reset=0 | State=01 | Light Output=001
Time=0        85000 | Reset=0 | State=10 | Light Output=010
Time=0        95000 | Reset=0 | State=00 | Light Output=100
```

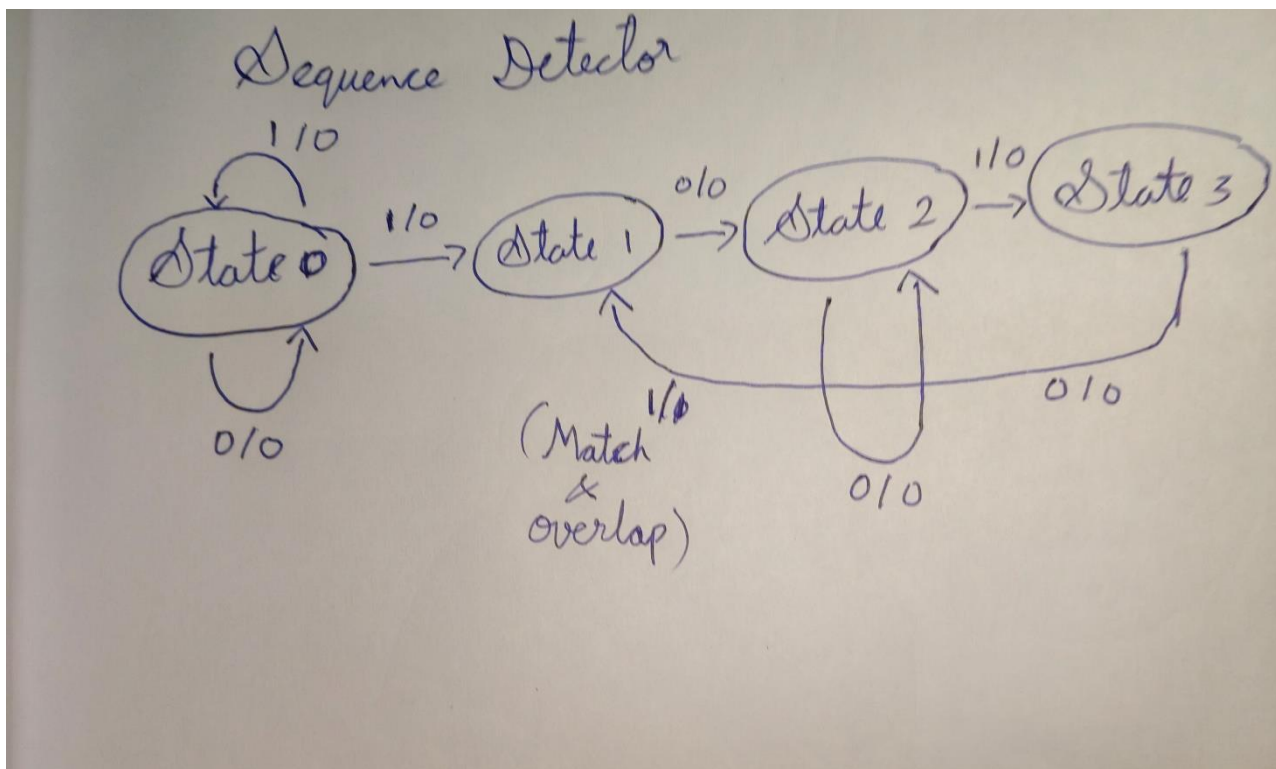


Waveform of Traffic Light Controller

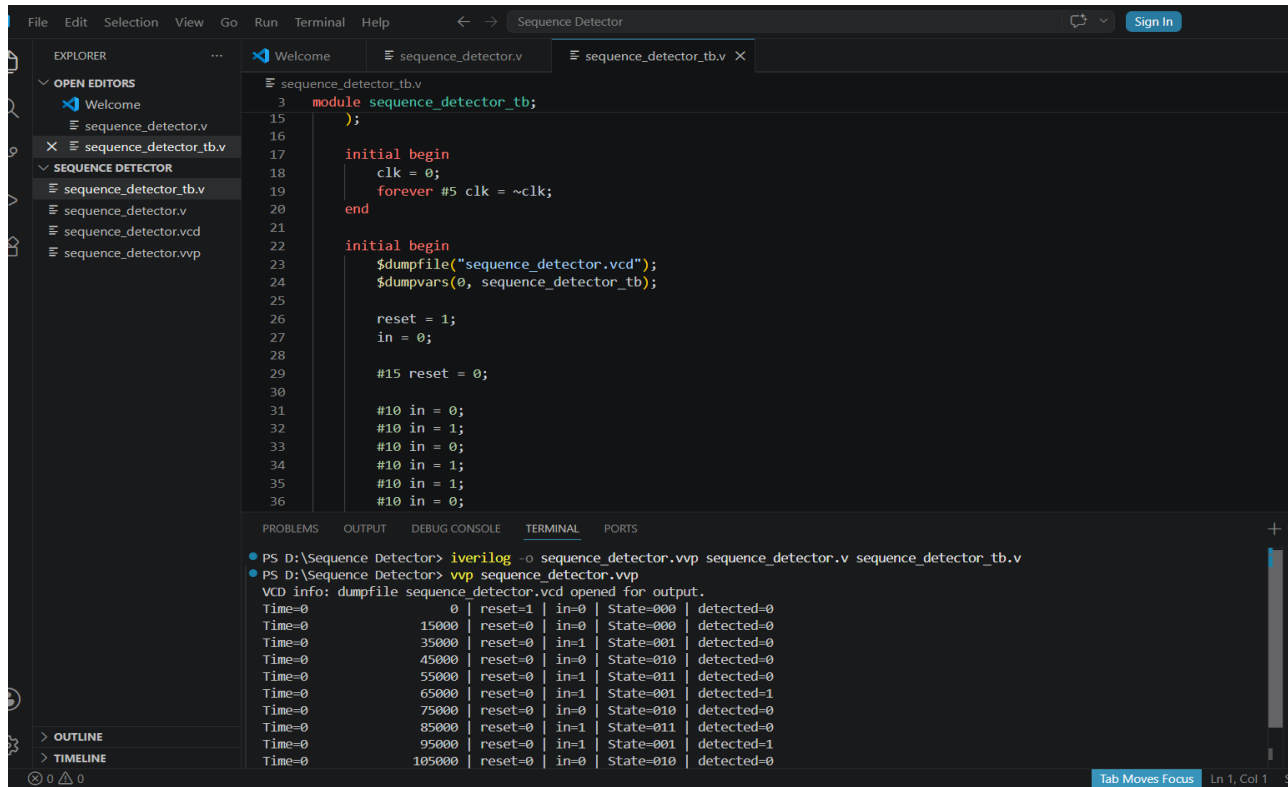


Sequence Detector

- A sequence detector is a type of sequential digital circuit designed to recognize a specific pattern of bits (0s and 1s) within a continuous stream of input data.
- The circuit has a finite number of "states." Each state represents how much of the target sequence has been successfully detected so far. For example, in your 1011 detector, State 0 means nothing matched, State 1 means "1" is matched, State 2 means "10" is matched, and so on.
- The circuit operates on a clock signal.
- With every active clock edge, it reads the current input bit and decides whether to move forward to the next state (if the bit matches the sequence) or fall back to an earlier state (if the sequence breaks).
- When the circuit reaches the final state that completes the target sequence, it drives an output signal high (usually to 1) for that clock cycle to signal a successful detection



Output of Sequence Detector



```
File Edit Selection View Go Run Terminal Help
sequence_detector_tb.v
sequence_detector.v
sequence_detector.vvp
sequence_detector.vcd
sequence_detector.v
```

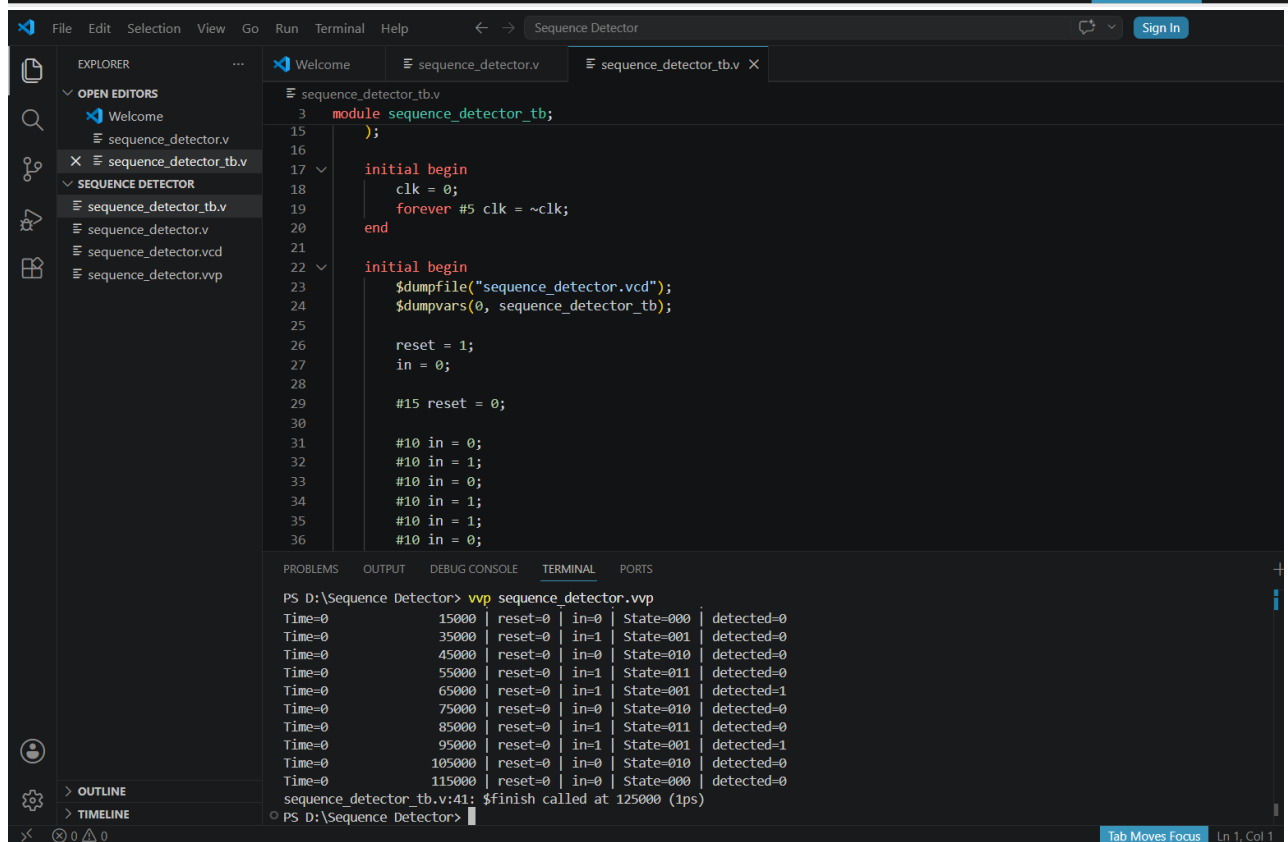
```
3 module sequence_detector_tb;
15 );
16
17 initial begin
18     clk = 0;
19     forever #5 clk = ~clk;
20 end
21
22 initial begin
23     $dumpfile("sequence_detector.vcd");
24     $dumpvars(0, sequence_detector_tb);
25
26     reset = 1;
27     in = 0;
28
29     #15 reset = 0;
30
31     #10 in = 0;
32     #10 in = 1;
33     #10 in = 0;
34     #10 in = 1;
35     #10 in = 1;
36     #10 in = 0;
```

PS D:\Sequence Detector> iverilog -o sequence_detector.vvp sequence_detector.v sequence_detector_tb.v

PS D:\Sequence Detector> vvp sequence_detector.vvp

VCD info: dumpfile sequence_detector.vcd opened for output.

Time=0	0	reset=1	in=0	State=000	detected=0
Time=0	15000	reset=0	in=0	State=000	detected=0
Time=0	35000	reset=0	in=1	State=001	detected=0
Time=0	45000	reset=0	in=0	State=010	detected=0
Time=0	55000	reset=0	in=1	State=011	detected=0
Time=0	65000	reset=0	in=1	State=001	detected=1
Time=0	75000	reset=0	in=0	State=010	detected=0
Time=0	85000	reset=0	in=1	State=011	detected=0
Time=0	95000	reset=0	in=1	State=001	detected=1
Time=0	105000	reset=0	in=0	State=010	detected=0



```
File Edit Selection View Go Run Terminal Help
sequence_detector_tb.v
sequence_detector.v
sequence_detector.vvp
sequence_detector.vcd
sequence_detector.v
```

```
3 module sequence_detector_tb;
15 );
16
17 initial begin
18     clk = 0;
19     forever #5 clk = ~clk;
20 end
21
22 initial begin
23     $dumpfile("sequence_detector.vcd");
24     $dumpvars(0, sequence_detector_tb);
25
26     reset = 1;
27     in = 0;
28
29     #15 reset = 0;
30
31     #10 in = 0;
32     #10 in = 1;
33     #10 in = 0;
34     #10 in = 1;
35     #10 in = 1;
36     #10 in = 0;
```

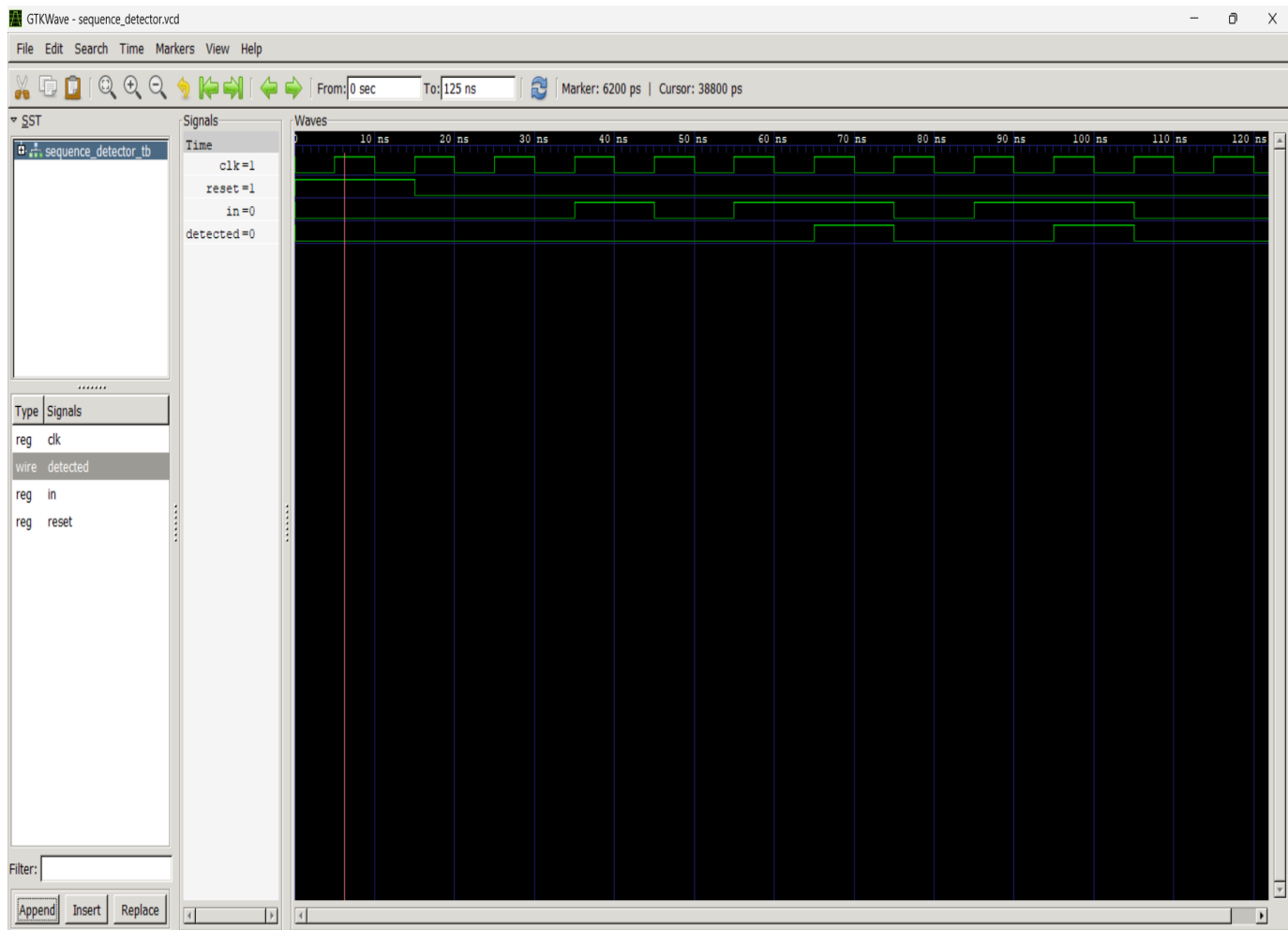
PS D:\Sequence Detector> vvp sequence_detector.vvp

Time=0	15000	reset=0	in=0	State=000	detected=0
Time=0	35000	reset=0	in=1	State=001	detected=0
Time=0	45000	reset=0	in=0	State=010	detected=0
Time=0	55000	reset=0	in=1	State=011	detected=0
Time=0	65000	reset=0	in=1	State=001	detected=1
Time=0	75000	reset=0	in=0	State=010	detected=0
Time=0	85000	reset=0	in=1	State=011	detected=0
Time=0	95000	reset=0	in=1	State=001	detected=1
Time=0	105000	reset=0	in=0	State=010	detected=0
Time=0	115000	reset=0	in=0	State=000	detected=0

sequence_detector_tb.v:41: \$finish called at 125000 (1ps)

PS D:\Sequence Detector>

Waveform of Sequence Detector



Observation

From this task, I have learnt about the Finite State Machine, Moore Finite State Machine, Mealy Finite State Machine, and its applications like Traffic Light Controller and Sequence Detector.

I also learnt about how to execute these circuits in Icarus Verilog, Visual Studio Code and to view its waveform using Gtkwave.